4 Cities (Rawalpindi, Islamabad, Peshawar, Lahore)

AQI Prediction System Report

Submitted by:

Fatima Muhummad Ali

10P Shine Internee Cohort 9 (Data sciences)

Project links

On

Streamlit | Github | Huggingface

1. Introduction

1.1 Background

Air pollution is a major environmental concern that can have significant effects on human health, quality of life, and the environment. One commonly used measure for communicating air pollution levels is the **Air Quality Index (AQI)**, which represents the overall level of air pollution based on the concentration of different pollutants. Higher AQI values indicate poorer air quality and potentially greater health risks.

Forecasting becomes increasingly difficult as the prediction horizon increases. While recent AQI observations can provide strong information about conditions in the next few hours or the following day, their predictive relationship with AQI several days into the future becomes weaker. This makes multi-horizon forecasting, particularly 24-, 48-, and 72-hour forecasting, a challenging machine-learning problem.

This project initially focused on five cities in Pakistan:

- Islamabad
- Lahore
- Peshawar
- Rawalpindi
- Karachi

These cities were selected to provide geographical diversity of the 5 major cities of Pakistan and to allow the forecasting models to be evaluated under different air-quality and environmental conditions.

1.2 Problem Statement

The objective of this project is to develop a machine-learning-based system capable of forecasting future AQI values using historical air-quality and weather observations.

More specifically, the system aims to predict AQI at three different forecasting horizons:

- **24 hours ahead**
- **48 hours ahead**
- **72 hours ahead**

The problem can therefore be formulated as:

Given historical AQI and weather observations up to the current time, predict the AQI for the corresponding future time at 24, 48, and 72 hours ahead.

Rather than developing a model that is evaluated only once on a fixed historical dataset, the project aims to develop a **continuous forecasting system**. New observations are periodically collected, processed, and incorporated into the system so that fresh predictions can be generated over time.

This requires more than simply training a machine-learning model. The complete system must also manage historical data, generate features, provide features to deployed models, store predictions, evaluate predictions against subsequently observed AQI values, monitor model performance, and support model retraining when performance deteriorates.

1.3 Project Objectives

The main objectives of the project are:

1. **Collect AQI and weather data** for the selected cities and construct a historical dataset suitable for time-series forecasting.
2. **Develop multi-horizon AQI forecasting models** capable of predicting AQI 24, 48, and 72 hours into the future.
3. **Compare different machine-learning approaches**, including Linear Regression, Ridge Regression, Random Forest, XGBoost, CatBoost, and LSTM.
4. **Investigate the effect of feature engineering** by introducing lag, rolling, temporal, weather, and AQI-derived features.
5. **Evaluate model performance** using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and coefficient of determination (R^2).
6. **Perform feature ablation experiments** to determine whether individual feature groups provide meaningful improvements in forecasting performance.
7. **Develop an end-to-end prediction pipeline** capable of periodically updating data, generating features, retrieving the required model inputs, and producing new AQI forecasts.
8. **Implement a feature store** to provide the latest engineered features consistently to the prediction system.
9. **Implement persistent data storage** for historical observations, predictions, and model-monitoring results.
10. **Monitor deployed model performance** by comparing predictions with the actual AQI values observed after the forecast horizon has passed.
11. **Implement an automated retraining workflow** that can identify underperforming models, retrain candidate models using newly accumulated data, and prepare improved model versions for deployment.

1.4 System Overview

The project was developed as an end-to-end AQI forecasting system rather than as an isolated machine-learning experiment. The overall workflow begins with the collection of AQI and weather observations from external data sources. These observations are stored as historical data and subsequently processed through the feature-engineering pipeline.

The **resulting features are used during both model training and prediction**. Multiple machine-learning models are trained and evaluated to determine the most suitable model for each forecasting horizon. The selected models are then made available to the production prediction pipeline.

For online prediction, the latest engineered features are retrieved through the feature-store layer and provided to the appropriate forecasting model. The generated predictions are stored in the database together with their target timestamps. **Once the target time has passed, the corresponding actual AQI value becomes available and is used to evaluate the prediction**.

The system also includes a monitoring component that tracks model performance over time. If a model's performance falls below predefined thresholds, the corresponding forecasting horizon can be flagged for retraining. The retraining pipeline then uses the latest historical data to train and evaluate new candidate models before preparing the selected model for deployment.

The high-level architecture can therefore be summarized as:

Data Sources → Historical Dataset → Feature Engineering → Model Training & Evaluation → Model Storage → Feature Store → Prediction Pipeline → Database → Monitoring → Retraining

The major components of the system are:

- **Data sources:** Provide AQI and weather observations.
- **Historical dataset:** Stores accumulated environmental observations used for training and evaluation.
- **Feature engineering:** Generates lag, rolling, temporal, weather, and derived AQI features.
- **Machine-learning models:** Generate forecasts for the 24-, 48-, and 72-hour horizons.
- **Model storage:** Stores trained model artifacts for retrieval by the prediction system.
- **Feature store:** Provides the latest engineered features required during inference.
- **Prediction pipeline:** Periodically generates new forecasts.
- **Database:** Stores historical observations, predictions, and monitoring results.

- **Monitoring pipeline:** Evaluates deployed models against subsequently observed AQI values.
- **Retraining pipeline:** Retrains models when monitoring indicates that performance has deteriorated.

Choice of Technology

Feature Store: Feast

The original recommendation for the project was to use *Hopsworks* as both the feature store and model registry. However, during the early development stage, issues were encountered with project configuration and naming. In addition, other project members experienced connection and free-tier limitations when working with Hopsworks. Since the system required a feature store that could operate reliably throughout development, testing, and deployment, an alternative was evaluated.

Feast was selected as the feature-store framework because it is open-source, lightweight, and can be run locally without depending on a proprietary cloud platform. It also provided the required feature-store functionality, including feature definitions, entities, feature views, offline data sources, and online feature retrieval.

This decision reduced dependency on a specific commercial platform while allowing the project to retain a proper feature-store architecture.

Database and Online Infrastructure: Supabase

Although Feast itself was selected as the feature-store framework, an online database was still required for persistent online feature serving and application data.

Supabase was therefore selected as the database and backend storage layer. It provided a practical free-tier PostgreSQL environment and could be integrated with the rest of the system without requiring a separately managed database server.

Supabase was used for multiple parts of the system rather than only storing application data. It hosted:

- Feast online feature data
- Historical/backfilled AQI data
- Prediction records generated by the forecasting pipeline
- Monitoring and model-performance records
- Data required by the Streamlit application to retrieve recent predictions

Using one persistent backend for these components simplified deployment and reduced the number of external services that needed to be maintained.

Model Registry: Hugging face

Hugging Face was selected for model storage and model version management. The project required a persistent location from which the deployed application and retraining pipeline could retrieve trained models.

Hugging Face provided a suitable free solution for storing the trained forecasting models and integrating model retrieval into the inference layer. The model registry contains the models associated with the three forecasting horizons:

- XGBoost for the 24-hour forecast
- CatBoost for the 48-hour forecast
- Random Forest for the 72-hour forecast

The model registry was separated from the feature-store infrastructure so that models could be updated independently of the feature-serving system.

Automation: GITHUB Actions

Automations used the suggested Github Actions in the project explanations.

Engineering Challenges and Solutions

1. Feast Timestamp Serialization Issue

One of the most significant implementation issues occurred while integrating Feast with the feature data.

The feature dataset used a `timestamp_utc` column representing the event time of each observation. The Parquet source correctly stored this value as a timezone-aware datetime:

```
datetime64[ns, UTC]
```

However, Feast's internal registry representation serialized event-time metadata using protobuf timestamp fields. These timestamps appeared internally as separate `seconds` and `nanos` values. During debugging, this created a mismatch between the timestamp representation expected by parts of the application and the representation returned by Feast.

Investigation

This became particularly problematic when timestamp values returned from the Feast online store were treated as normal datetime strings. Instead of receiving a directly usable ISO-

formatted timestamp, the application could encounter an integer timestamp representation, resulting in datetime parsing errors.

The Parquet data itself was verified to contain valid timezone-aware timestamps, and the individual feature groups were also tested directly through Feast. These tests confirmed that the feature values themselves were being stored and retrieved correctly. The problem was therefore isolated to timestamp handling rather than feature generation or model inference.

Solution

Timestamp normalization was consequently introduced at the application boundaries, with timestamps explicitly converted to **UTC-aware pandas timestamps** rather than relying on implicit conversion.

2. Prediction History

The hourly pipeline generates predictions continuously. Initially, the prediction data could not simply be treated as a single "latest prediction" because each execution produces forecasts for different target times and forecasting horizons.

Solution

By creating a Predictions table in Supabase that stores the predictions. Every pipeline execution stores:

- prediction time
- target time
- city
- forecast horizon
- predicted AQI
- model used

This allows historical predictions to remain available for later evaluation.

For example, a prediction generated at time `T` for the 24-hour horizon has a target time of `T + 24 hours`. Once the actual AQI for that target time becomes available, the prediction can be evaluated against it.

This design also supports the monitoring and retraining pipelines because historical predictions are required to calculate model errors and detect degradation.

3. Model and Feature Consistency

Another issue encountered during development was maintaining consistency between the features used during training and those supplied during inference.

Solution

The inference layer therefore uses an explicit **feature-column definition containing the same features expected by the trained models**. This prevents accidental inclusion of metadata such as timestamps or city names and ensures that the model receives the expected feature structure.

The final inference feature set contains environmental, weather, lag, rolling, temporal, difference, and encoded-city features.

4. Explainability and SHAP Performance

SHAP analysis introduced a separate performance challenge. Initially, SHAP calculations were performed over a larger dataset than necessary, causing the Streamlit application to spend significant time calculating explanations even when the user had not requested them.

Solution

The solution was to make **SHAP analysis conditional and restrict its calculation to the selected city and forecasting horizon**. This reduced unnecessary computation and prevented explainability calculations from slowing down normal dashboard usage.

2. Data Collection and Preparation

2.1 Data Collection

The first stage of the project involved collecting historical air-quality and weather data for the selected cities. The initial dataset consisted of five cities in Pakistan:

- Islamabad
- Rawalpindi
- Lahore
- Karachi
- Peshawar

The geographical coordinates of each city were used when requesting the data from the APIs. Data was collected at an hourly resolution so that the resulting dataset could support short- and medium-term AQI forecasting.

The project used the **Open-Meteo Air Quality API** to obtain historical air-pollution observations and the **Open-Meteo Historical Weather API** to obtain corresponding weather observations.

The initial data collection scripts were kept separately in the `api_scripts` directory:

```
api_scripts/
├── api_call_air_data.py
├── api_call_weather.py
└── merge_script.py
```

This separation allowed the air-quality and weather data to be collected independently before being combined into a single dataset.

2.1.1 Historical Air-Quality Data

Historical air-quality observations were retrieved using the *Open-Meteo Air Quality API.* The API requests were made using the latitude and longitude of each city and covered approximately two years of historical data.

The initial collection period was defined dynamically using the current date:

- **End date:** Previous day
- **Start date:** Approximately 730 days before the end date
- **Frequency:** Hourly
- **Timezone:** UTC

The following air-quality variables were initially collected:

Variable	Description
PM2.5	Fine particulate matter
PM10	Particulate matter
CO	Carbon monoxide
NO_2	Nitrogen dioxide
SO_2	Sulphur dioxide
O_3	Ozone
US AQI	United States Air Quality Index
European AQI	European Air Quality Index

The collected data was converted into a Pandas DataFrame. Each observation was associated with its corresponding city, geographical coordinates, and UTC timestamp.

The timestamp was explicitly converted into a UTC-aware datetime format to maintain consistent time handling throughout the project.

2.1.2 Historical Weather Data

Weather observations were collected separately using the *Open-Meteo Historical Weather API*. The weather data was retrieved for the same cities and historical period so that each AQI observation could be associated with the environmental conditions occurring at approximately the same time.

The weather variables collected included:

- Temperature
- Relative humidity
- Wind speed
- Wind direction
- Surface pressure
- Precipitation
- Precipitation
- Cloud cover

The exact set of weather variables was refined during the feature-engineering stage based on their availability and usefulness for forecasting.

2.2 Initial Data Storage

During the initial development stage, the collected data was stored locally as CSV files. This provided a simple and convenient format for exploratory analysis, preprocessing, and experimentation.

The initial project structure contained separate files for air-quality and weather observations:

```
data/
├── historical_air_quality_4_cities.csv
├── historical_weather_4_cities.csv
└── combined_air_weather_4_cities.csv
```

Although the files were initially named `4_cities`, the original data-collection process included **five cities**, including Karachi. *Karachi was subsequently excluded from the final modelling dataset after model evaluation showed substantially weaker and less consistent forecasting performance, hurting overall performance.*

The separate AQI and weather datasets were combined using the city and timestamp as the basis for matching observations. This produced a unified dataset containing both air-quality and weather information.

The resulting combined dataset formed the basis for the exploratory data analysis and subsequent feature-engineering experiments.

2.3 Data Merging

After collecting the two datasets, the air-quality and weather observations were merged into a single dataset.

The merge was performed using:

- **City**
- **Timestamp**

This ensured that the weather conditions corresponded to the same city and time as the AQI observation.

The resulting dataset contained the historical observations required for both exploratory analysis and machine learning experiments.

2.4 Exploratory Data Analysis

Before model development, exploratory data analysis was performed to understand the structure and characteristics of the dataset.

The EDA was conducted using the notebook:

```
EDA/
└── eda.ipynb
```

Findings

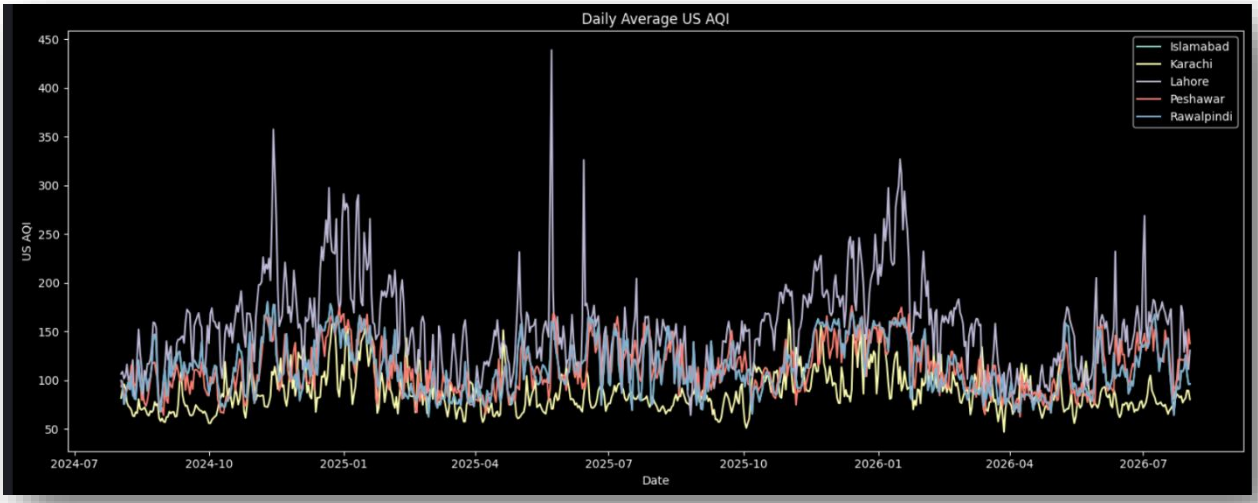
AQI and Pollution Levels

- AQI shows substantial variation across the dataset, indicating that air quality is not constant and changes considerably over time.
- **PM2.5 is one of the most important pollution variables**, showing a strong relationship with AQI. This is expected because fine particulate matter is a major contributor to overall air-quality degradation.
- Other pollutants, including PM10, NO_2, SO_2, CO, and O_3, also show variation and provide additional information for predicting AQI.

- The presence of high-AQI observations indicates periods of severe pollution, making the dataset suitable for forecasting changing air-quality conditions rather than only average conditions.

Temporal Patterns

- AQI exhibits noticeable temporal variation across hours, days, and months.
- Pollution levels are not uniformly distributed throughout the year, suggesting the presence of **seasonal effects**.
- The temporal patterns justify incorporating features such as **hour, day of week, and month** into the prediction models.
- Because the data is time-dependent, a chronological train-test split is more appropriate than a random split, since random splitting could allow information from future observations to influence training.

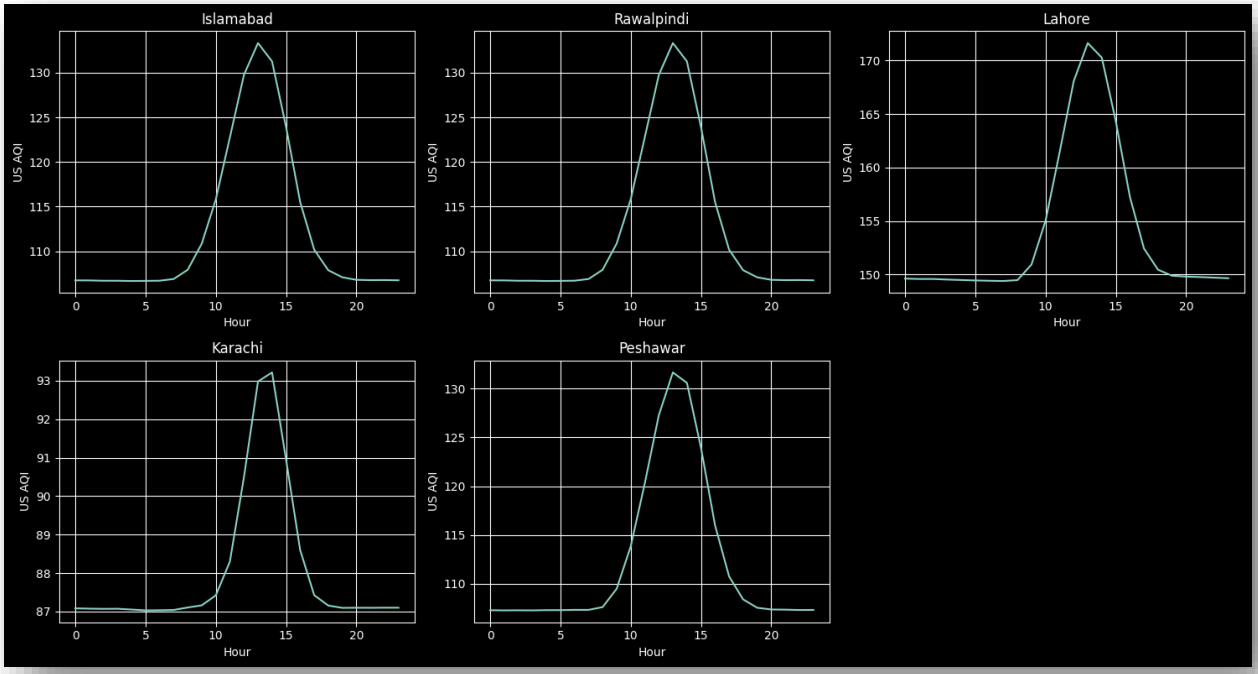


AQI over the course of 2 years

City-Level Differences

- The cities do not exhibit identical AQI behaviour. Differences in pollution levels and their temporal patterns indicate that **location is an important predictive feature**.
- Lahore generally exhibits higher pollution levels than the less polluted periods observed in cities such as Islamabad, while Peshawar and Rawalpindi also show their own variations.

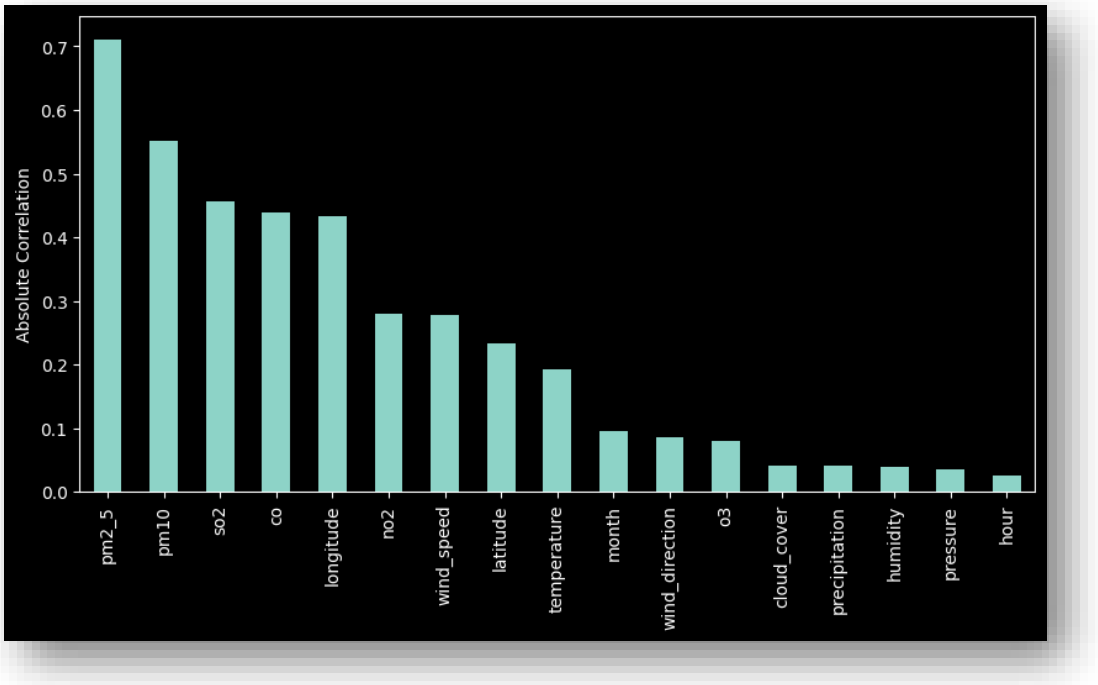
- These differences support including the city as an encoded model feature rather than treating all observations as coming from a single location.
- While the figures at first may look the same, the difference is in their y-axis reading is apparent.



Average AQI by city in a day

Weather Relationships

- Weather variables such as **temperature, humidity, pressure, wind speed, precipitation, and cloud cover** vary alongside pollution levels.
- Wind and precipitation are particularly relevant because they can influence the dispersion or removal of airborne pollutants.
- The relationship between weather and AQI is not necessarily linear, supporting the use of tree-based machine-learning models capable of capturing nonlinear relationships.



Absolute Correlation

Data Quality and Modeling Implications

- The dataset contains multiple interacting environmental variables, making AQI prediction a multivariate forecasting problem.
- Extreme pollution observations are important rather than simply being treated as noise, since these periods represent the conditions for which an AQI forecasting system is particularly useful.
- The EDA therefore supports using **pollutant, meteorological, temporal, and city-level features together** for prediction.
- The observed temporal and city-level differences also justify the use of separate forecasting horizons for **24-hour, 48-hour, and 72-hour predictions**.

Overall Finding

The EDA indicates that **AQI is influenced by a combination of pollutant concentrations, meteorological conditions, location, and temporal patterns**. The considerable variation between cities and across time suggests that a model using only historical AQI would be insufficient. Combining environmental and temporal features provides a stronger basis for forecasting future AQI and supports the machine-learning approach used in the project.

2.5 Data Preprocessing

Before the data could be used for machine-learning experiments, several preprocessing steps were performed.

These included:

- Converting timestamps to a consistent datetime format
- Using UTC as the internal time standard
- Sorting observations chronologically
- Handling duplicate city/timestamp observations
- Aligning AQI and weather observations
- Handling missing values
- Separating input features from forecasting targets

The data was maintained chronologically because AQI forecasting is a time-series problem. Randomly shuffling the observations would not accurately represent the conditions under which the deployed system would make future predictions.

2.6 Forecast Target Construction

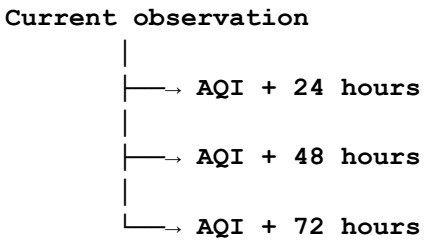
To transform the historical time-series data into a supervised-learning problem, future AQI values were used as prediction targets.

Three forecasting targets were created:

- `target_24h`
- `target_48h`
- `target_72h`

This allowed the same historical observations to be used to construct three separate forecasting problems.

For example:



This target construction allowed the project to evaluate whether different machine-learning approaches were capable of forecasting AQI at progressively longer horizons.

2.7 Chronological Train-Test Splitting

Because the project deals with time-series forecasting, the dataset was divided into training and testing sets using a **timestamp-based chronological split** rather than randomly dividing the observations.

The training data therefore consisted of earlier observations, while the test data consisted of later observations.

This approach better represents the real-world forecasting scenario, where a model is trained using historical observations and then asked to predict future observations that it has not previously seen.

Issue Encountered: Timestamp-Based Data Leakage

An initial approach divided the dataset according to an 80% row-count boundary. Because observations were recorded hourly, this could result in observations from the same date or time period appearing on both sides of the boundary.

This was undesirable for a time-series forecasting problem because the test set should represent a genuinely later period than the training data.

The splitting procedure was therefore changed to use an explicit **timestamp boundary**.

This ensured that:

Training period < Test period

The timestamp-based approach provided a more realistic evaluation of how the forecasting models would perform on future data.

2.8 Evolution of Data Storage

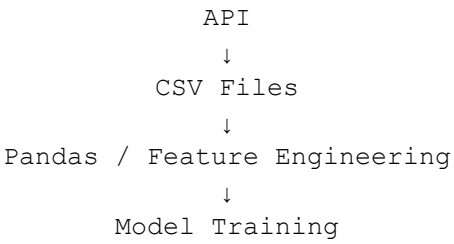
The project's data-storage architecture evolved as the system progressed from experimentation to deployment.

During the initial modelling stage, CSV files were sufficient for storing historical observations and generated datasets. However, a continuously operating forecasting system required persistent and remotely accessible storage.

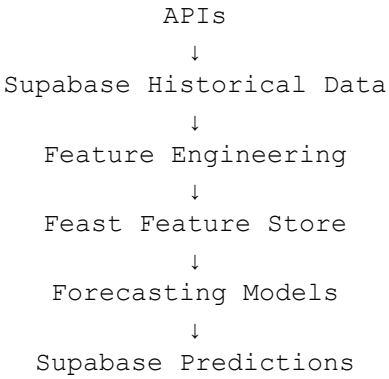
The architecture was therefore later expanded to use **Supabase PostgreSQL** for persistent application data.

The storage architecture eventually became:

Initial Development:



Production System:



The transition from local CSV files to database-backed storage allowed the system to support continuous data updates, prediction storage, monitoring, and automated retraining.

3. Model Development and Experimental Evaluation

3.1 Summary

The model development process was carried out incrementally. Models were first trained using the initial dataset without additional feature engineering to establish a baseline. **Lag-based AQI features** were then introduced, followed **by temporal, rolling, and rate-of-change features**.

LSTM models were also evaluated to determine whether a sequence-based neural network could better capture the temporal behaviour of AQI.

The models were evaluated using **MAE, RMSE, and R^2**, with lower MAE and RMSE indicating better performance and higher R^2 indicating better explanatory performance.

3.2 Initial Models Without Feature Engineering

The first experiment used the available AQI and weather variables without introducing additional engineered features. The purpose was to establish a baseline for comparing the effect of later feature-engineering techniques.

Initial Results

Rank	Model	MAE ↓	RMSE ↓	R^2 ↑
1	**CatBoost**	**12.33**	**17.53**	**0.7431**
2	**XGBoost**	**12.55**	**18.03**	**0.7284**
3	**Random Forest**	**13.32**	**19.29**	**0.6892**
4	Ridge Regression	19.81	25.76	0.4454
5	Linear Regression	19.81	25.76	0.4454

Even without additional feature engineering, the tree-based models produced substantially better results than the linear models. *CatBoost* achieved the lowest MAE, while Linear and Ridge Regression showed considerably higher errors.

This suggested that the relationship between the available environmental variables and AQI was not purely linear. *Though, this wasn't as the final product despite the number because it was only restricted to one horizon that of 24-hour. The result of 48-hour and 72-hour was of course substantially worse.*

4. Lag Feature Engineering

The next experiment introduced **lag features** to explicitly provide the models with information about previous AQI observations.

The reasoning was that AQI has strong temporal dependence: the AQI observed recently is likely to contain useful information about AQI in the near future. Lag Feature had the strongest effect

The following lag features were created independently for each city:

- `aqi_lag_1`
- `aqi_lag_3`
- `aqi_lag_6`
- `aqi_lag_12`
- `aqi_lag_24`

These represent AQI values from 1, 3, 6, 12, and 24 hours earlier.

The data was first sorted by city and timestamp, ensuring that lag values were calculated within each city rather than across different cities.

4.1 Lag Feature Results

Horizon	Model	MAE ↓	RMSE ↓	R^2 ↑
24h	Linear Regression	13.766	19.250	0.691
	Ridge	13.766	19.250	0.691
	Random Forest	**13.017**	**18.219**	**0.723**
	XGBoost	12.588	17.782	0.736
	CatBoost	**12.411**	**17.125**	**0.755**
48h	Linear Regression	**18.123**	**24.305**	**0.507**
	Ridge	19.816	26.155	**0.507**
	Random Forest	18.500	24.566	0.496
	XGBoost	18.260	24.727	0.490
	CatBoost	18.348	24.775	0.488
72h	Linear Regression	**19.816**	**26.155**	**0.429**
	Ridge	19.816	26.155	**0.429**
	Random Forest	21.861	28.339	0.330
	XGBoost	20.739	27.600	0.364
	CatBoost	**20.387**	**27.175**	**0.384**

The lag features demonstrated that recent AQI history was an important source of predictive information. In particular, *the short-term forecasting task benefited strongly from information about previous AQI observations.*

This provided motivation for developing a richer set of temporal and AQI-derived features.

5. LSTM Experiment

Because AQI is a time-dependent variable, an LSTM model was also evaluated. LSTM networks are designed to process sequences and can potentially learn temporal dependencies that may not be captured explicitly by traditional machine-learning models.

A sequence length of **24 hours** was used for the LSTM. The data was divided chronologically, scaled, converted into sequences, and then passed to the LSTM model.

6.1 Initial LSTM Results

Forecast Horizon	MAE ↓	RMSE ↓	R^2 ↑
24 hours	16.58	21.99	0.597
48 hours	18.95	25.12	0.475
72 hours	20.96	27.63	0.360

The initial LSTM results were weaker than those obtained from the best traditional machine-learning models, particularly for the 24-hour forecast.

The LSTM was therefore retained as a comparison model rather than assuming that a more complex neural architecture would necessarily produce better forecasts.

6. Extended Feature Engineering

Following the lag-feature experiment, additional temporal and statistical features were introduced.

7.1 Rolling AQI Features

Rolling statistics were calculated using previous AQI observations:

- 3-hour rolling mean
- 24-hour rolling mean
- 24-hour rolling minimum
- 24-hour rolling maximum

A one-hour shift was applied before calculating the rolling statistics:

```
Previous AQI observations
        ↓
    shift(1)
        ↓
Rolling calculation
        ↓
    Feature
```

This ensured that the rolling features did not include the current/future target value and therefore reduced the risk of target leakage.

7.2 Time-Based Features

The following temporal features were added:

- Hour of day
- Day of month
- Day of week
- Month
- Hour sine
- Hour cosine
- Weekend indicator
- Rush-hour indicator

Cyclical encoding was used for the hour of day so that the relationship between the end and beginning of a daily cycle could be represented more naturally.

6.3 AQI Rate-of-Change Features

Two additional features were introduced:

- **`aqi_diff_1h`**
- **`aqi_diff_24h`**

These represent the change in AQI relative to one hour and 24 hours earlier.

They provide the models with information about whether AQI is increasing or decreasing rather than only its absolute value.

6.4 Final Feature-Engineering Function

The complete feature-engineering process therefore followed:

Raw AQI + Weather Data

↓

Lag Features

↓

Rolling Features

↓

Temporal Features

↓

AQI Rate-of-Change Features

↓

Engineered Dataset

The complete feature set was subsequently used for model comparison, ablation testing, and final model training.

7. Feature-Engineered Model Results

After introducing the additional temporal, rolling, and AQI-derived features, the models were retrained and evaluated.

Horizon	Model	MAE ↓	RMSE ↓	R^2 ↑
24h	Random Forest	**12.93**	**18.04**	**0.728**
	XGBoost	12.95	18.28	0.721
	LSTM	15.76	21.47	0.617
	CatBoost	20.10	26.51	0.414
48h	**LSTM**	**15.33**	**20.72**	**0.643**
	Ridge	17.92	24.15	0.513
	Random Forest	18.21	24.34	0.505
	CatBoost	18.36	24.51	0.499
	XGBoost	18.28	24.77	0.488
72h	**LSTM**	**15.86**	**21.13**	**0.629**
	CatBoost	18.36	24.51	0.499

	Ridge	19.71	25.98	0.437
	XGBoost	20.38	27.04	0.390
	Random Forest	21.56	28.21	0.336

The additional features did not produce a consistent improvement for every model and horizon. However, they provided useful information about recent AQI behaviour, temporal patterns, and environmental conditions and were retained for further evaluation.

The results also demonstrated that the relative performance of model architectures could change substantially after feature engineering. This reinforced the need to compare models under a common evaluation procedure rather than selecting a model based on a single experiment.

8. City-Level Evaluation

Overall metrics alone can hide important differences between locations. Therefore, model performance was also evaluated separately for each city.

This analysis was particularly important because AQI dynamics can differ between cities due to differences in pollution sources, weather conditions, urban structure, and temporal patterns.

Example: 72-Hour LSTM Performance

City	R^2
Lahore	**0.784**
Islamabad	0.515
Rawalpindi	0.452
Peshawar	0.418
Karachi	**-0.106**

Lahore showed the strongest predictive performance in this analysis, while Karachi was substantially more difficult to predict.

The negative R^2 for Karachi indicated that the model was unable to explain its AQI variation effectively under the tested setup. Further analysis showed that Karachi also had strong short-term AQI autocorrelation, with the correlation decreasing as the forecasting horizon increased.

Karachi Dataset Decision

Karachi was initially included in the project, but its consistently weaker forecasting performance and different AQI dynamics made it unsuitable for the final modelling scope. Hence, it was removed to maintain the model's performance.

Given the project's scope, the final modelling dataset was therefore restricted to:

- Islamabad
- Lahore
- Peshawar
- Rawalpindi

This decision was made after evaluation rather than being imposed before model development.

9. AQI-Range Error Analysis

Model performance was also evaluated according to the actual AQI range to determine whether prediction accuracy changed under different pollution levels.

AQI Range	Samples	MAE ↓
51–100	7,057	15.27
101–150	4,443	15.81
151–200	2,151	18.18
201–300	154	30.53
300+	17	15.20

The results indicate that prediction errors generally increased as AQI became higher. *The 201–300 range had substantially higher MAE than the lower ranges.*

However, the extreme ranges contain relatively few observations. In particular, the 300+ category contains only 17 samples, so its performance should not be interpreted as statistically representative.

Overall, the analysis suggests that the dataset provides substantially more information about normal and moderately polluted conditions than extreme pollution events.

10. Feature Ablation Testing

After developing the complete feature set, feature ablation experiments were performed to determine whether individual feature groups were providing consistent improvements.

The experiments involved removing selected groups of features and retraining the models.

The feature groups considered included:

- AQI lag features
- Rolling AQI features
- Temporal features
- Weather features
- AQI rate-of-change features

The purpose was to determine whether a smaller feature set could achieve equal or better performance while reducing unnecessary inputs.

10.1 Ablation Findings

The ablation experiments produced *only marginal and inconsistent changes in MAE, RMSE, and R^2 across the forecasting horizons.*

No reduced feature set consistently outperformed the complete engineered feature set across all three horizons.

Therefore, the complete engineered feature set was retained for the final models.

This decision also provided the final models with access to multiple sources of information, including recent AQI behaviour, longer-term AQI statistics, temporal patterns, and weather conditions.

11. Final Model Evaluation

Following the previous experiments, the final models were selected using the standardized evaluation procedure.

The final comparison included the traditional machine-learning models and LSTM using the final modelling dataset and engineered features.

Horizon	Model	MAE ↓	RMSE ↓	R^2 ↑
24h	LSTM	13.32	18.01	0.745
	Random Forest	12.59	18.15	0.741
	XGBoost	**12.18**	**17.53**	**0.759**
	CatBoost	12.30	17.56	0.758
48h	LSTM	16.24	20.77	0.661

	Random Forest	16.04	20.92	0.656
	XGBoost	**15.84**	21.02	0.653
	CatBoost	**15.84**	**20.71**	**0.663**
72h	LSTM	27.20	32.42	0.174
	Random Forest	**16.03**	**21.16**	**0.648**
	XGBoost	16.46	22.13	0.615
	CatBoost	16.07	21.27	0.644

11.1 Final Selected Models

Based primarily on MAE, with R^2 and RMSE considered as supporting metrics, the final production models were selected independently for each forecasting horizon.

Horizon	Best Model	MAE	RMSE	R2
24h	XGBoost	**12.18**	**17.53**	0.759
48h	CatBoost	**15.84**	**20.71**	**0.663**
72h	Random Forest	**16.07**	**21.27**	**0.648**

The final results demonstrate that no single model architecture was optimal across all forecasting horizons. XGBoost achieved the best 24-hour performance, CatBoost provided the strongest 48-hour performance, and Random Forest achieved the best 72-hour performance.

This horizon-specific model selection was therefore used in the production system:

24h → XGBoost

48h → CatBoost

72h → Random Forest

The results also demonstrate the expected reduction in forecasting accuracy as the prediction horizon increases. The 24-hour model achieved an R^2 of 0.759, while the 72-hour model achieved an R^2 of 0.648. Despite this degradation, the 72-hour model remained capable of explaining a substantial portion of the observed AQI variation.

12. Feature Store Selection and Implementation

12.1 Feast

Feast was particularly useful for the AQI forecasting system because the same engineered features used during model training needed to be available when generating live predictions.

The feature-store architecture was designed around:

Historical AQI + Weather Data

↓

Feature Engineering

↓

Feast Feature Definitions

↓

Online Feature Store

↓

Prediction Pipeline

This prevented the prediction system from having to independently reconstruct every feature whenever a prediction was requested.

12.3 Feast Architecture

The project used Feast to define and manage the features required by the forecasting models.

The feature set included:

- AQI lag features
- Rolling AQI statistics
- AQI rate-of-change features
- Weather variables
- Temporal features
- City information

The Feast repository was maintained separately under the `feast_st` directory and contained the feature definitions and Feast configuration.

The initial implementation used a **local SQLite online store** during development and testing.

The architecture was therefore initially:

Feature Engineering → Feast → SQLite Online Store → Prediction

After the prediction pipeline was stabilized, the online store was moved to **Supabase PostgreSQL** to make the system more suitable for cloud deployment.

The final architecture became:

Feature Engineering → Feast → Supabase PostgreSQL → Prediction

This allowed the feature store to remain logically separate from the prediction application while using a remotely accessible database rather than a local file.

13. Problems Encountered During Feast Implementation

Although Feast provided the required feature-store functionality, its integration introduced several implementation challenges.

13.1 Entity DataFrame Conflict

One issue occurred while testing Feast's online feature retrieval.

The error originated inside Feast's internal:

```
_prepare_entities_to_read_from_online_store()
```

function, before the online store was actually queried.

The problem was related to the representation of the entity data supplied to Feast. The online-store implementation expected entities in a particular internal structure, with support for a list of dictionaries that could then be converted into a columnar representation.

The issue was resolved by correcting the entity input structure used during feature retrieval.

13.2 Missing Features in the Online Store

A second issue occurred when Feast successfully executed the feature retrieval request but returned `None` values for the requested features.

The problem was not with the feature definitions themselves. The required feature records had simply **not yet been materialized into the online store**.

The solution was to run Feast's incremental materialization process. This populated the online store with the latest feature values from the feature source.

After materialization, the prediction system was able to successfully retrieve the required:

- AQI features
- Weather features
- Lag features
- Rolling features
- Temporal features

for the selected city.

13.3 Timestamp Representation Issue

Another issue occurred because timestamps were represented differently between the feature source and the online store.

The local SQLite implementation stored timestamps internally as integer Unix timestamps, while parts of the application expected timezone-aware datetime values.

This resulted in errors when the returned timestamp was treated as an ISO-formatted datetime string.

Resolution

The issue was resolved by explicitly converting the returned Unix timestamp into a UTC-aware datetime and avoiding unnecessary retrieval of timestamp fields when they were not required as model features.

14.4 Migration to Supabase PostgreSQL

Once the Feast pipeline was working locally, the local SQLite online store became unsuitable for the final cloud architecture.

A local SQLite database would not provide a reliable shared online feature store for a remotely deployed Streamlit application and automated GitHub Actions workflows.

The online store was therefore migrated to **Supabase PostgreSQL**.

The resulting architecture allowed:

- GitHub Actions to update features remotely.
- Streamlit to retrieve features remotely.
- Multiple pipeline executions to access the same feature store.
- Feature data to persist independently of the execution environment.

This was an important step in converting the project from a locally executed machine-learning pipeline into a continuously running cloud-based prediction system.

14. Cloud Storage and Model Management

14.1 Supabase Database

Supabase was selected as the project's central PostgreSQL database. It provides a remotely accessible database that can be used by the Streamlit application as well as the automated GitHub Actions pipelines.

The project uses four main application tables.

Historical Data Table

The `historical_data` table stores the observed AQI and weather data used throughout the system.

It contains:

- Timestamp and city information
- Geographic coordinates
- Pollutant measurements such as PM2.5, PM10, CO, NO_2, SO_2, and O_3
- US AQI

- Weather variables including temperature, humidity, pressure, wind, precipitation, and cloud cover

This table acts as the continuously growing historical dataset. New observations collected by the prediction and monitoring pipelines are added to it over time.

Predictions Table

The `predictions` table stores every forecast generated by the production system.

Each record contains:

- Prediction timestamp
- Target timestamp
- City
- Forecast horizon
- Predicted AQI
- Model used
- Actual AQI, once available
- Percentage error
- Evaluation status

Initially, `actual_aqi` is `NULL` and `evaluated` is `FALSE`. Once the target time has passed, the daily monitoring pipeline retrieves the actual AQI and updates the prediction record.

This table therefore provides the link between **what the model predicted** and **what actually occurred**.

Monitoring Table

The `monitoring` table stores the results of model-performance monitoring.

For each city and forecast horizon, it records:

- Number of predictions
- Number of bad predictions
- Consecutive bad days
- MAE
- Baseline MAE
- Flagged status
- Monitoring date

The `flagged` field is used by the weekly retraining pipeline to determine whether a model requires retraining.

Database Role in the System

The four tables therefore serve different purposes:

Table	Purpose
`historical_data`	Stores observed AQI and weather history
`predictions`	Stores model forecasts and their eventual actual values
`monitoring`	Stores model-performance monitoring results
Feast online-store table	Stores the latest features required for inference

This separation keeps **raw observations, predictions, monitoring information, and online features** logically independent.

15. Hugging Face Model Storage

The final production models are stored in the Hugging Face repository:

`flork-18115/AQI_prediciton_models`

The application does not store these models locally. Instead, the model registry uses `hf_hub_download()` to retrieve the required model from Hugging Face when it is needed.

The model is then loaded according to its format:

- `.pkl` → loaded using Joblib
- `.cbm` → loaded using CatBoost

This approach keeps the trained models separate from the application code and allows new model versions to be uploaded independently.

Hugging Face was also useful for the automated retraining pipeline because newly trained models can be uploaded remotely after retraining, while the currently deployed model remains

explicitly defined in the model registry until the new model has been reviewed and selected for deployment.

17. Hourly AQI Prediction Pipeline

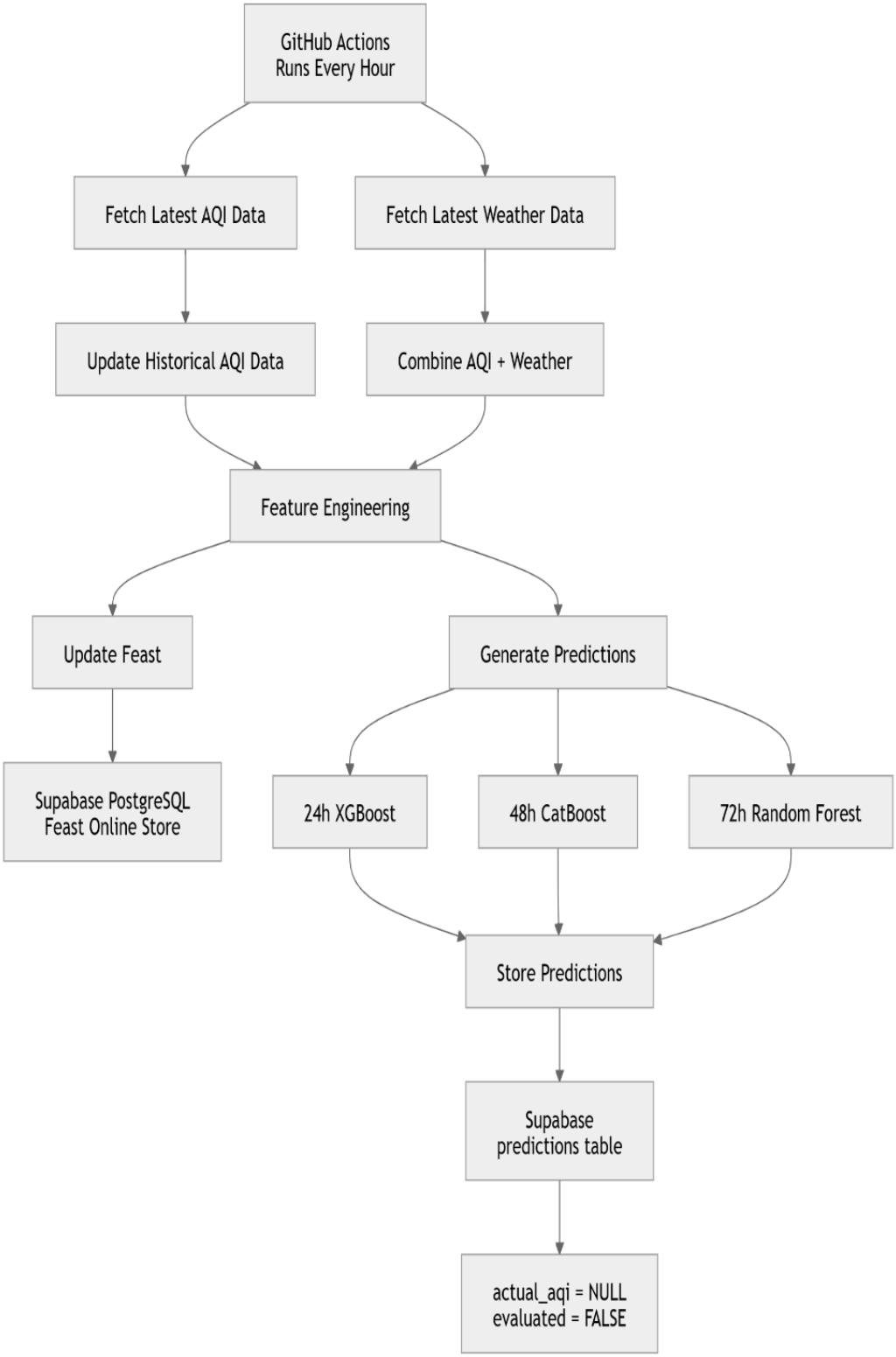
This is the **main production pipeline**. Its job is to continuously update the latest AQI data, generate new features, update Feast, and produce fresh **24h, 48h, and 72h predictions**.

17.1 Flow

- **GitHub Actions**
  - Runs the pipeline **every hour**.
  - Can also be triggered manually using `workflow_dispatch`.
- **Fetch latest data**
  - Gets the latest **AQI/air-pollution data** and **weather data** for:
    - Islamabad
    - Lahore
    - Peshawar
    - Rawalpindi
- **Update historical data**
  - The newly received observed AQI data is added to the historical dataset.
  - Duplicate city/timestamp records are removed.
- **Feature Engineering**
  - Creates the required AQI features such as:
    - lag features
    - rolling statistics
    - time-based features
    - city encoding
  - Combines these with the latest weather information.
- **Update Feast**
  - The newly generated feature data is passed to **Feast**.
  - Feast's online store in **Supabase/PostgreSQL** is updated.
- **Generate predictions**
  - Each city receives:
    - **24-hour → XGBoost**
    - **48-hour → CatBoost**
    - **72-hour → Random Forest**
- **Store predictions**
  - Each prediction is inserted into the **`predictions` Supabase table**.
  - It initially has:
    - `predicted_aqi`

- `actual_aqi = NULL`
- `evaluated = FALSE`

The important point is that **this pipeline is concerned with prediction**, not model performance monitoring.



18. Daily Monitoring and Evaluation Pipeline

The Daily Monitoring and Evaluation Pipeline is responsible for evaluating the performance of the deployed AQI forecasting models after their predictions have reached their target times.

Purpose

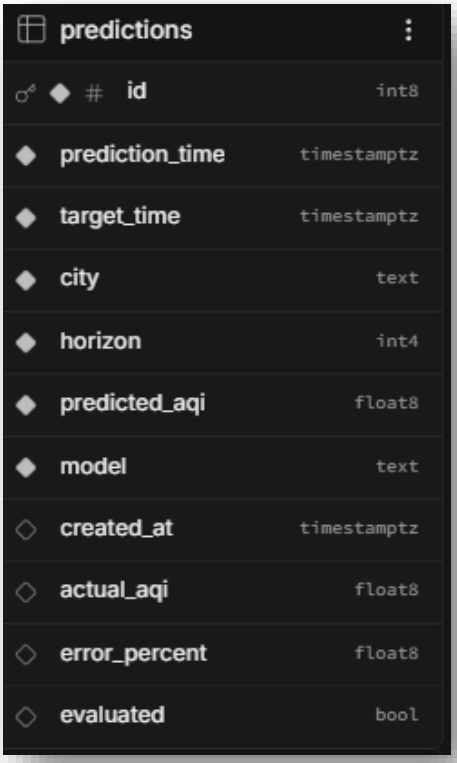
"This system is more robust, daily retraining can be computationally expensive and if the metrics are lower for the new model the system could perform progressively worse than before. This daily monitoring collects data over time giving the model a chance to prove themselves. If a model provides consecutive bad predictions, it is flagged and a new model is trained in the weekly retraining pipeline and replaces it."

The pipeline is executed automatically **once per day through GitHub Actions**. It retrieves unevaluated predictions from the Supabase `predictions` table whose target time has already passed and evaluates them against the corresponding observed AQI.

18.1 Prediction Evaluation

The pipeline first identifies predictions that are ready for evaluation. A prediction is considered ready when its target timestamp has passed and its `evaluated` field is still set to `FALSE`.

For each pending prediction, the system retrieves the corresponding actual AQI using historical air-quality data from **Open-Meteo**. The actual value is then written back to the corresponding record in the Supabase `predictions` table.



Schema of Predictions table from Supabase

The system calculates the prediction error and marks the prediction as evaluated. This allows each forecast to be linked with its eventual observed value and ensures that the same prediction is not evaluated repeatedly.

If the required actual AQI or weather observation is temporarily unavailable, the prediction is left unevaluated. It can therefore be processed again during a subsequent monitoring run rather than being permanently lost.

18.2 Updating the Historical Dataset

The monitoring pipeline also contributes to the continuous growth of the project's historical dataset. Newly available actual AQI observations and their corresponding weather observations are added to the Supabase `historical_data` table.

This means that the historical dataset is not limited to the data originally collected for model development. As the production system operates, new observations are continuously accumulated and can later be used for model retraining.

18.3 Model Performance Monitoring

After completed predictions have been evaluated, the pipeline monitors model performance for each **city and forecasting horizon**.

The system calculates the **Mean Absolute Error (MAE)** from the evaluated predictions and compares the observed performance against the established baseline. Daily prediction errors are also used to identify periods of consistently poor performance.

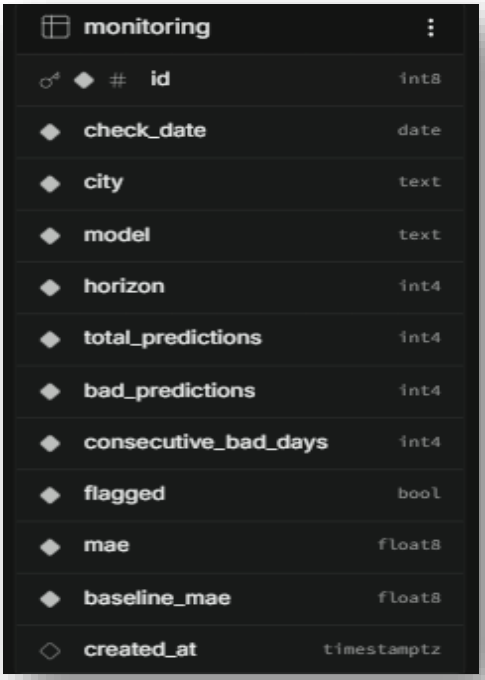
The monitoring process performs its weekly assessment on **Monday**, using the previous completed Monday-to-Sunday period. This provides a consistent evaluation window while allowing the system to continue evaluating individual predictions every day.

18.4 Retraining Flag Detection

The monitoring system is designed to identify when a deployed model may require retraining. A model and forecasting horizon can be flagged when predefined performance conditions are exceeded.

The current criteria include **five or more consecutive bad days** or a **30-day/monitoring MAE exceeding the baseline MAE by 20%**.

When a model is flagged, the result is recorded in the Supabase `monitoring` table along with the relevant MAE, baseline MAE, number of predictions, bad predictions, consecutive bad days, city, model, and forecasting horizon.



Schema of Monitoring table in Supabase

The monitoring pipeline does not perform retraining itself. Instead, it acts as the **performance-detection stage** of the system. Its output provides the weekly retraining pipeline with the information required to determine which model or forecasting horizon requires further training


GitHub Actions
Every Day
daily_monitoring.py
Find Pending Predictions
target_time passed
evaluated = FALSE
Group by City
backfill_api.py
Open-Meteo
Historical AQI
Open-Meteo Archive API
Historical Weather
Match Actual AQI
to Prediction
Match Actual Weather
to Timestamp
Update Supabase
predictions
Store in
historical_data
actual_aqi
error_percent
evaluated = TRUE
Is Monday?
No
Yes
Finish Daily Monitoring
Evaluate Previous
Monday-Sunday
Calculate MAE
Calculate Bad Days
Calculate Consecutive
Bad Days
Flag Conditions
5+ Consecutive Bad Days
MAE > Baseline × 1.15
Flag Model
No Flag
Insert Result into
monitoring table
Manual Retraining
if Flagged

19. Weekly AQI Model Retraining Pipeline

The Weekly AQI Model Retraining Pipeline is responsible for identifying underperforming forecasting models, retraining them using the latest available historical data, evaluating multiple model architectures, and publishing the newly selected model to the Hugging Face Model Hub. The pipeline is designed to retrain only the forecast horizons that have been flagged by the monitoring system rather than unnecessarily retraining every model each week.

The workflow is executed automatically every Monday through **GitHub Actions**. It begins by checking the Supabase `monitoring` table for models or horizons that have been flagged for retraining. If no horizons are flagged, the workflow terminates without performing any training. If one or more horizons are flagged, the pipeline retrieves the historical data required for retraining.

19.1 Monitoring Flag Check

The first stage queries Supabase to determine whether any forecasting horizons have been marked as requiring retraining.

The system supports three prediction horizons:

- **24-hour**
- **48-hour**
- **72-hour**

Only horizons where `flagged = True` are processed. This makes the retraining process targeted and reduces unnecessary computational work.

If no flagged horizons are found, the pipeline stops immediately.

19.2 Historical Data Retrieval

For every flagged horizon, the pipeline retrieves the available historical AQI and weather observations from the Supabase `historical_data` table.

This dataset contains actual environmental observations rather than model predictions. It therefore provides the basis for generating new training examples and allows the models to learn from the latest data accumulated since their previous training cycle.

The historical data is loaded into memory and passed to the feature engineering stage.

19.3 Feature Engineering

Before model training, the raw historical data is transformed into a feature-rich dataset using the project's feature engineering process.

The feature engineering stage generates several types of time-series features:

- **Lag features**
  - 1-hour
  - 3-hour
  - 6-hour
  - 12-hour
  - 24-hour
- **Rolling statistics**
  - 3-hour moving averages
  - 24-hour minimum
  - 24-hour maximum
  - 24-hour mean
- **Temporal features**
  - Hour of day
  - Day of week
  - Month
  - Cyclical sine/cosine representations
  - Weekend indicators
  - Peak-hour indicators
- **Rate-of-change features**
  - 1-hour AQI difference
  - 24-hour AQI difference

The rolling features use shifted values so that information from the prediction target does not leak into the training features. The resulting engineered dataset is then passed to the model-training stages.

19.4 Dual Model Retraining

The pipeline uses two different modelling approaches for each flagged horizon.

Traditional Machine Learning

The engineered dataset is passed through the ML preprocessing pipeline and used to train multiple candidate models:

- Ridge Regression

- Random Forest
- XGBoost
- CatBoost

Each model is evaluated on the test set using:

- **MAE**
- **RMSE**
- **R^2**

The best-performing ML model is selected based on MAE. This produces the **Best ML Model** for that particular forecasting horizon.

LSTM

The same historical data is also passed independently to the LSTM pipeline.

The LSTM does not rely on the ML preprocessing pipeline. Instead, it performs its own:

- Target preparation
- Sequence generation
- Chronological train/test split
- Scaling
- Time-series preparation

A sequence length of **24 hours** is used for the LSTM. The model is then trained and evaluated using MAE, RMSE, and R^2.

This separation allows the traditional ML models and the LSTM to use preprocessing approaches appropriate to their respective architectures.

19.5 Final Model Comparison

After training is complete, the pipeline has two candidates:

1. The best-performing traditional ML model
2. The newly trained LSTM

These two candidates are compared directly.

The primary selection metric is **Mean Absolute Error (MAE)**. The model with the lower MAE becomes the final winner for that forecasting horizon.

If both models have the same MAE, the pipeline uses **R^2 as the tie-breaker**, selecting the model with the higher R^2.

The winning model, along with its evaluation metrics and predictions, is returned for deployment preparation.

19.6 Model Storage

Once a final winner has been selected, the model is saved locally in the GitHub Actions environment.

Models are stored using a timestamped naming convention based on the training date and forecasting horizon.

For example:

```
2026-08-29_24.pkl
2026-08-29_48.cbm
2026-08-29_72.keras
```

The file format depends on the selected architecture:

- LSTM → `.keras`
- CatBoost → `.cbm`
- Other ML models → `.pkl`

This provides a versioned model artifact that can be uploaded and retained independently of the source code.

19.7. Hugging Face Upload

The newly trained winning model is then uploaded to the project's Hugging Face Model Hub repository.

The upload uses the `HF_TOKEN` stored securely as a GitHub Actions secret. The pipeline also includes retry handling to account for temporary network or service failures.

The current implementation allows up to **three upload attempts**, with a short delay between failed attempts.

If all attempts fail, the upload is considered unsuccessful and the pipeline does not proceed with the monitoring cleanup.

19.8 Monitoring Flag Cleanup

The monitoring flag is removed only after the new model has been successfully uploaded to Hugging Face.

This provides an important safeguard: a failed model upload does not make the system believe that retraining and deployment preparation were successfully completed.

The resulting automated lifecycle is therefore:

Flag detected → Retrain → Evaluate → Select winner → Save → Upload → Clear flag

The human review/approval process occurs **outside this automated pipeline**, after the model has been successfully uploaded.

19.9 GitHub Actions Scheduling

The weekly retraining process is executed using a dedicated GitHub Actions workflow.

It is scheduled for:

Monday at 01:17 UTC (06:17 PKT)

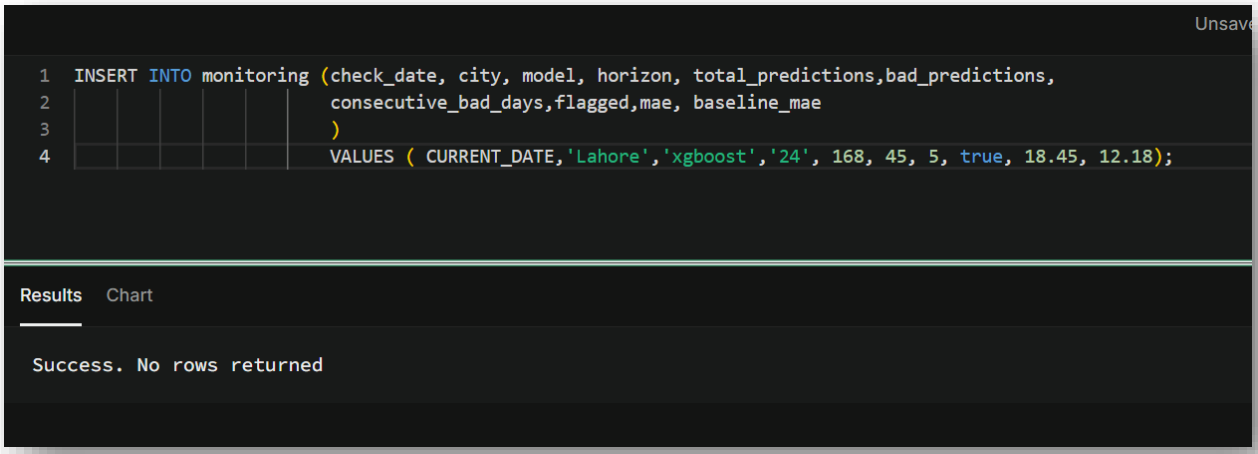
The workflow:

1. Checks out the repository.
2. Sets up Python 3.11.
3. Installs project dependencies.
4. Loads the required GitHub secrets.
5. Executes `weekly_retraining.py`.

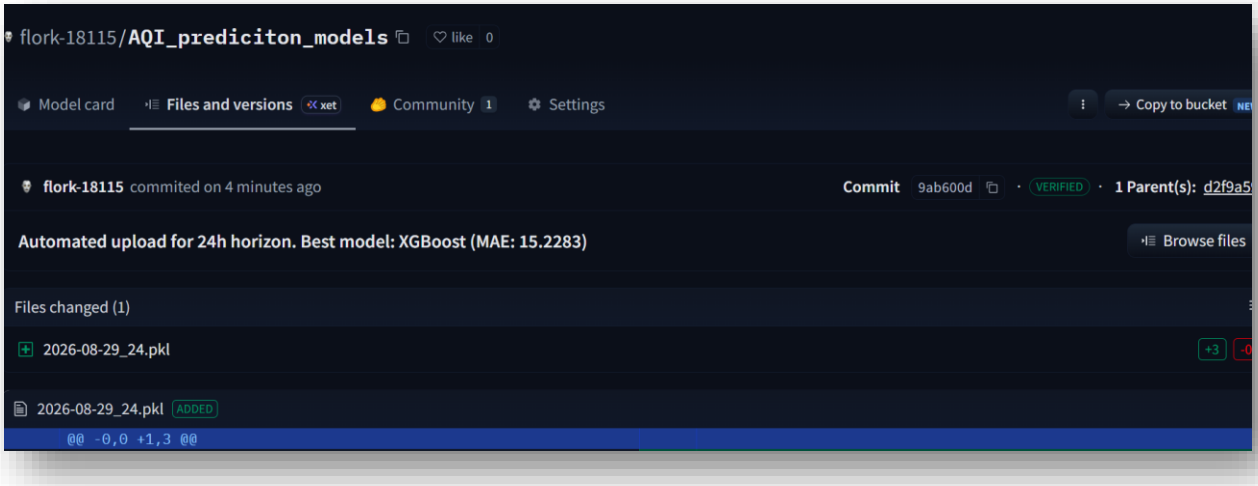
This scheduling allows the weekly monitoring process to complete first and gives the retraining pipeline access to the latest monitoring results.

Overall Pipeline

The weekly retraining system therefore forms the model improvement stage of the larger AQI prediction architecture. Daily monitoring identifies when a deployed model is no longer performing within acceptable limits, while the weekly retraining pipeline responds to those flags by searching for a better model using the latest historical observations.

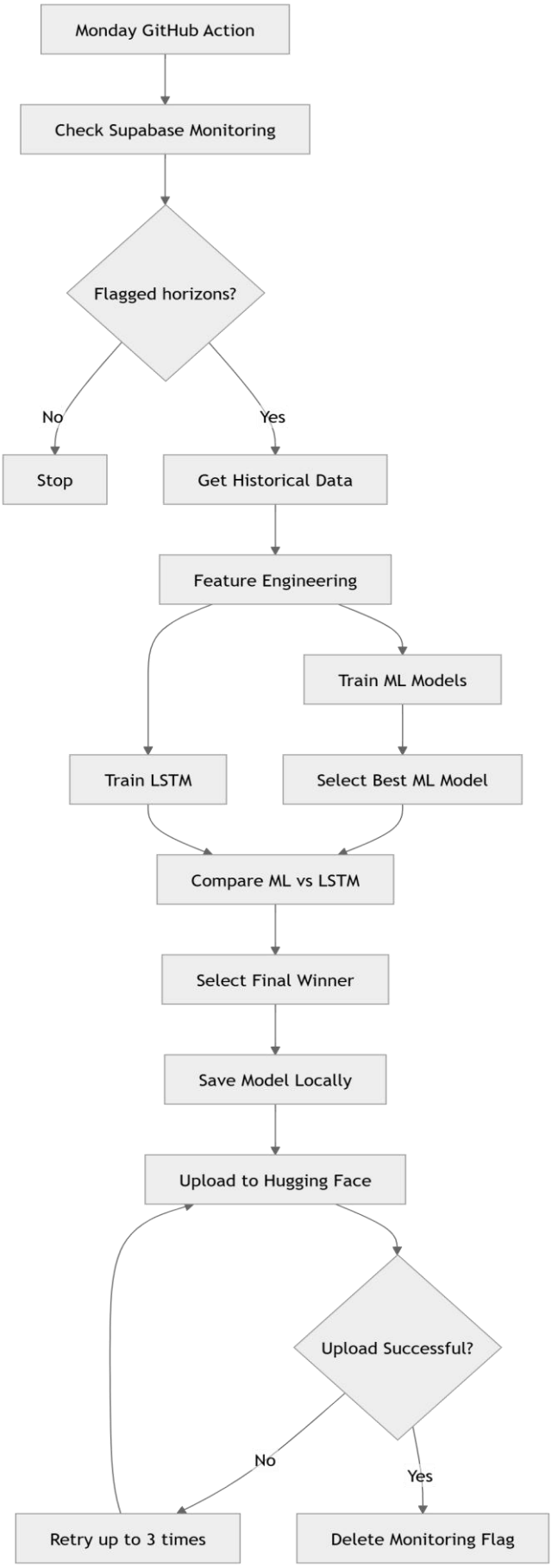


Dummy Data added for a run when a model is Flagged



Retrained model added in Hugging face

Flow Diagram


Monday GitHub Action
Check Supabase Monitoring
Flagged horizons?
No
Yes
Stop
Get Historical Data
Feature Engineering
Train ML Models
Train LSTM
Select Best ML Model
Compare ML vs LSTM
Select Final Winner
Save Model Locally
Upload to Hugging Face
Upload Successful?
No
Yes
Retry up to 3 times
Delete Monitoring Flag

Decision to Not Automate Model Replacement

The final step of automatically replacing the deployed production model with a newly retrained version was deliberately kept as *a manual process due to severe and unsolvable integration errors encountered during automation testing*. When the automated model replacement mechanism was implemented, it consistently *corrupted the state of dependent pipelines causing cascading failures in other pipelines especially the hourly pipeline where the model is used every hour to generate inferences,* that proved impossible to resolve without destabilizing the entire system.

Therefore, a manual deployment approach was adopted as a safer and more reliable strategy. The automated retraining pipeline continues to handle all training, evaluation, and model selection tasks, saving the winning model to the Hugging Face repository with *a timestamped filename and a descriptive commit message that includes the model's MAE, type, and city*. This allows a human operator to easily review the model's performance metrics and safely perform the simple, low-risk replacement of the production model.

20. Streamlit Dashboard Workflow and SHAP

The Streamlit application serves as the primary user interface for the AQI Prediction System, providing real-time access to forecasts, model performance monitoring, and model explainability features.

20.1 Application Architecture

The application follows a modular structure with three main components:

1. **app.py** – The main entry point that orchestrates the dashboard layout and user interactions.
2. **streamlit_pred.py** – Handles all Supabase data retrieval operations for predictions, monitoring data, and backfill status.
3. **streamlit_shap.py** – Manages SHAP-based model explainability, loading models from Hugging Face and generating feature importance visualizations from SHAP.py file /models.

20.2 User Workflow

1. City Selection & Prediction Retrieval

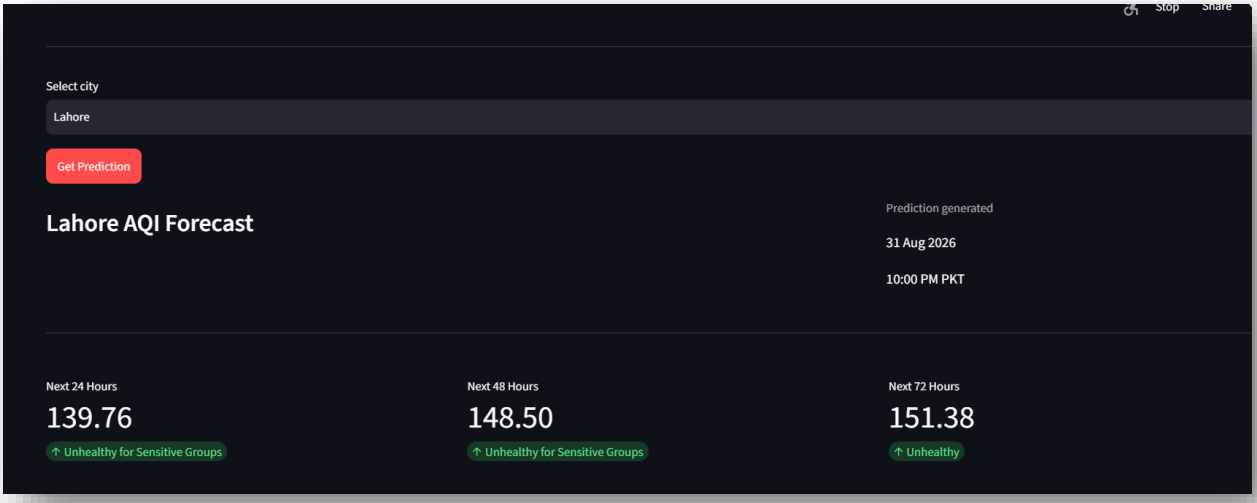
Upon selecting a city from the dropdown menu (Islamabad, Lahore, Peshawar, or Rawalpindi) and clicking the "Get Prediction" button, the application queries the Supabase predictions table

for the latest forecast. The system verifies that all three prediction horizons (24h, 48h, and 72h) are available before displaying results. Retrieved predictions are cached for 5 minutes using @st.cache_data to minimize database queries.

2. Forecast Visualization

The dashboard presents predictions through:

- **AQI Metric Cards** showing predicted values
- **Hazardous Alerts** are shown when the AQI exceeds a certain limit
- **Trend Line Chart** displaying the forecast trajectory across the three horizons
- Prediction generation timestamp displayed in PKT timezone



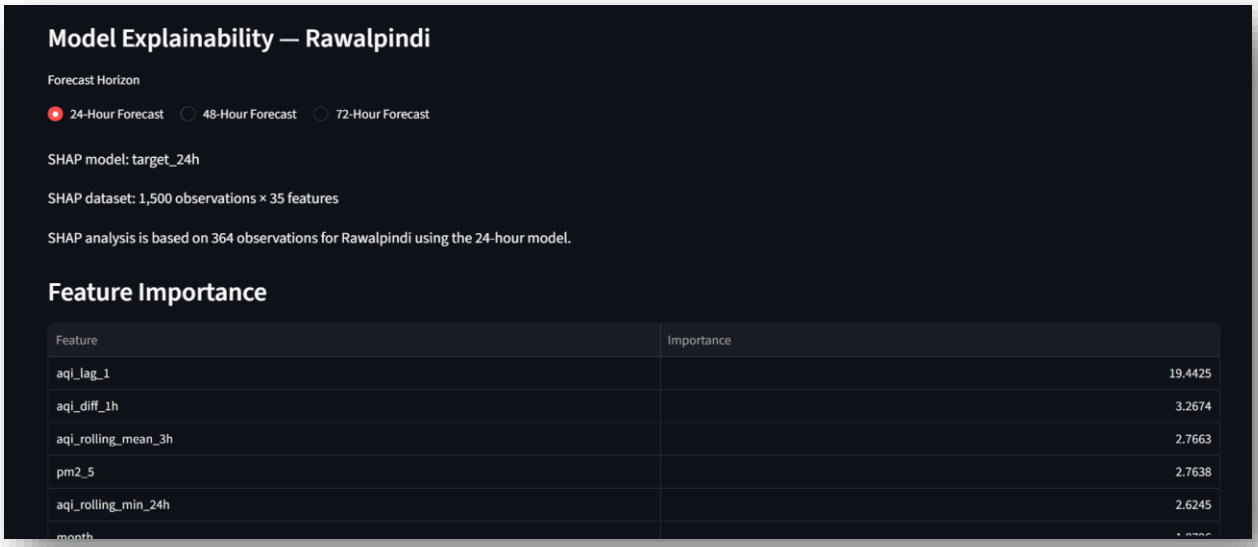
Prediction Dashboard

3. Model Explainability (SHAP Analysis)

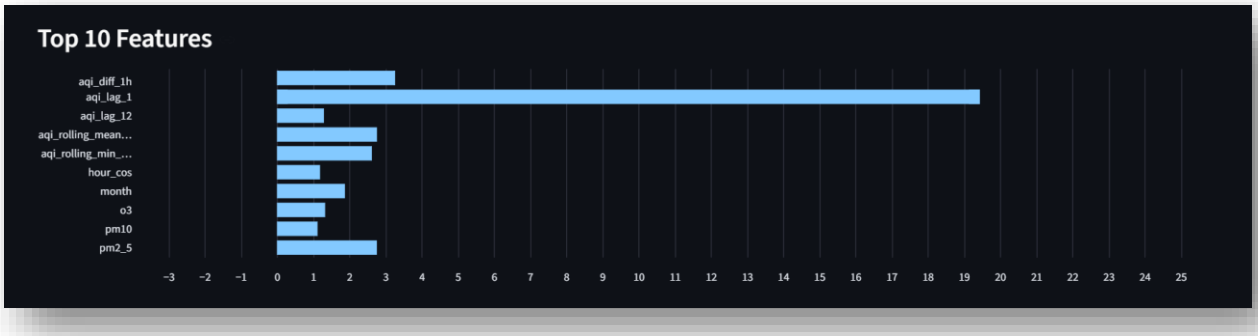
For any displayed prediction, users can explore model behavior through:

- **Horizon Selection** – Toggle between 24h, 48h, and 72h forecasts
- **Feature Importance Table** – Top 10 features ranked by mean absolute SHAP values
- **Bar Chart** – Visual representation of feature importance
- **Summary Plot** – SHAP summary plot showing feature influence distribution
- **Dependence Plot** – Interactive visualization of individual feature effects

The SHAP analysis loads the appropriate model from Hugging Face *(flork-18115/AQI_prediciton_models)* and samples up to 500 test observations for the selected city to compute explanations.



SHAP Explanations



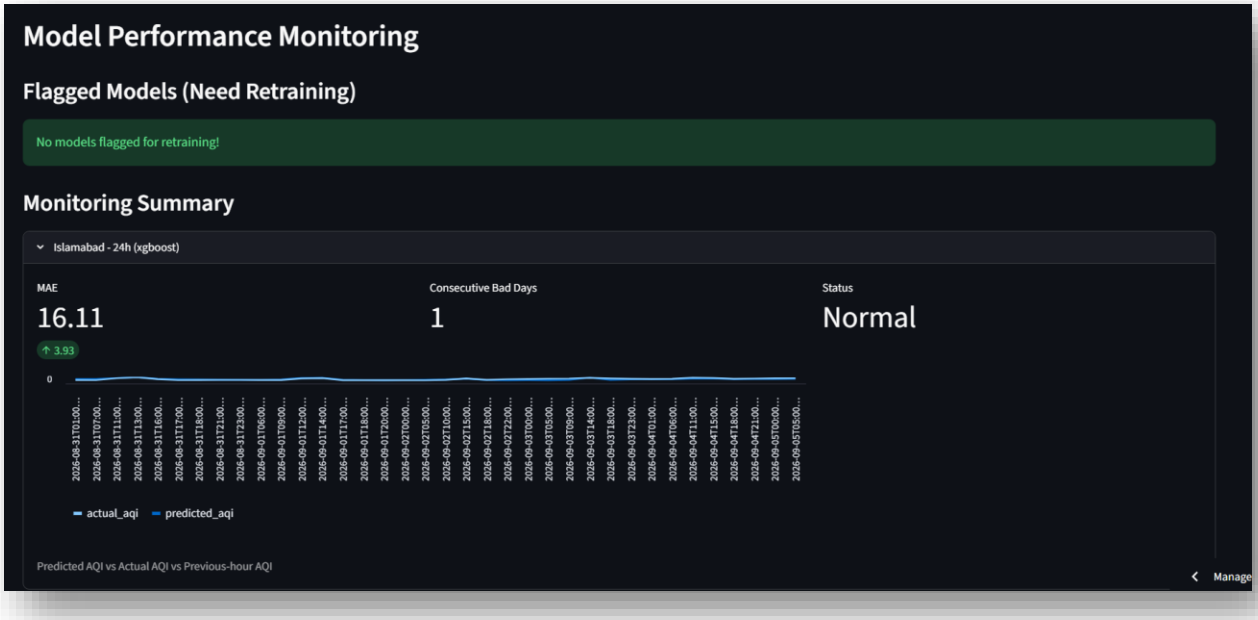
Feature Importance Graph

4. Performance Monitoring Dashboard

The monitoring section displays:

- **Flagged Models**: Expanders showing models identified for retraining, with MAE, consecutive bad days, and weekly bad prediction counts

- **Monitoring Summary**: Latest performance metrics for each city-horizon combination, including MAE deviation from baseline
- **Historical Prediction Charts**: Line charts comparing predicted AQI, actual AQI, and previous-hour AQI for the last 100 evaluated predictions
- **Monitoring Summary:** Shows the days the model gave prediction more the Baseline MAE. The green arrow pointing above, shows difference between actual and predicted. The **Status is Normal** though, models with more than **5 consecutive bad days on the same city** will be marked for retraining. This ensures each model gets enough time and opportunity. Better models remain while the worse ones are retrained on more data. This also prevents **continuous and unnecessary training**.

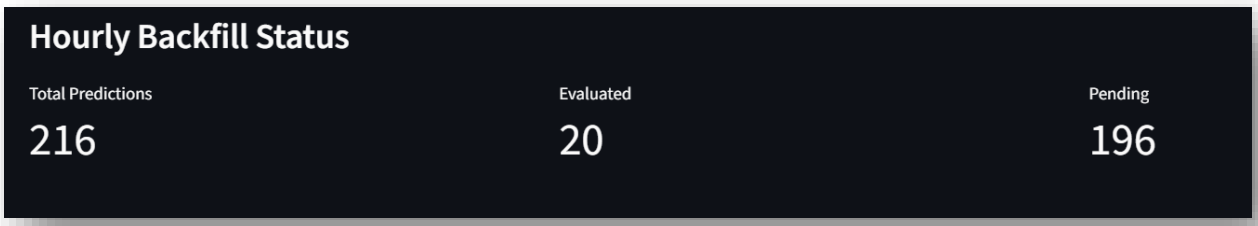


Model Performance Section and Summary

Actual VS Predicted Graph on Dashboard

5. System Status

A backfill status section shows the total number of predictions stored, how many have been evaluated against actual observations, and pending evaluations.



Predictions Evaluated Display

20.3 Data Caching Strategy

The application implements a tiered caching strategy:

- @st.cache_data(ttl=300) for database queries (predictions, monitoring data, historical data)
- @st.cache_resource for loading machine learning models from Hugging Face
- This ensures responsive performance while maintaining data freshness

20.4 Integrations

- **Supabase** – Central data source for predictions, monitoring metrics, and historical observations
- **Hugging Face Hub** – Remote model storage for XGBoost (24h), CatBoost (48h), and Random Forest (72h) models
- **SHAP** – TreeExplainer for XGBoost and Random Forest, custom ShapValues implementation for CatBoost

20.5 Deployment

The application is deployed as a Streamlit cloud service, with secrets managing Supabase credentials and Hugging Face tokens, providing a production-ready interface for stakeholders to monitor air quality forecasts and model health.

21. Concerns

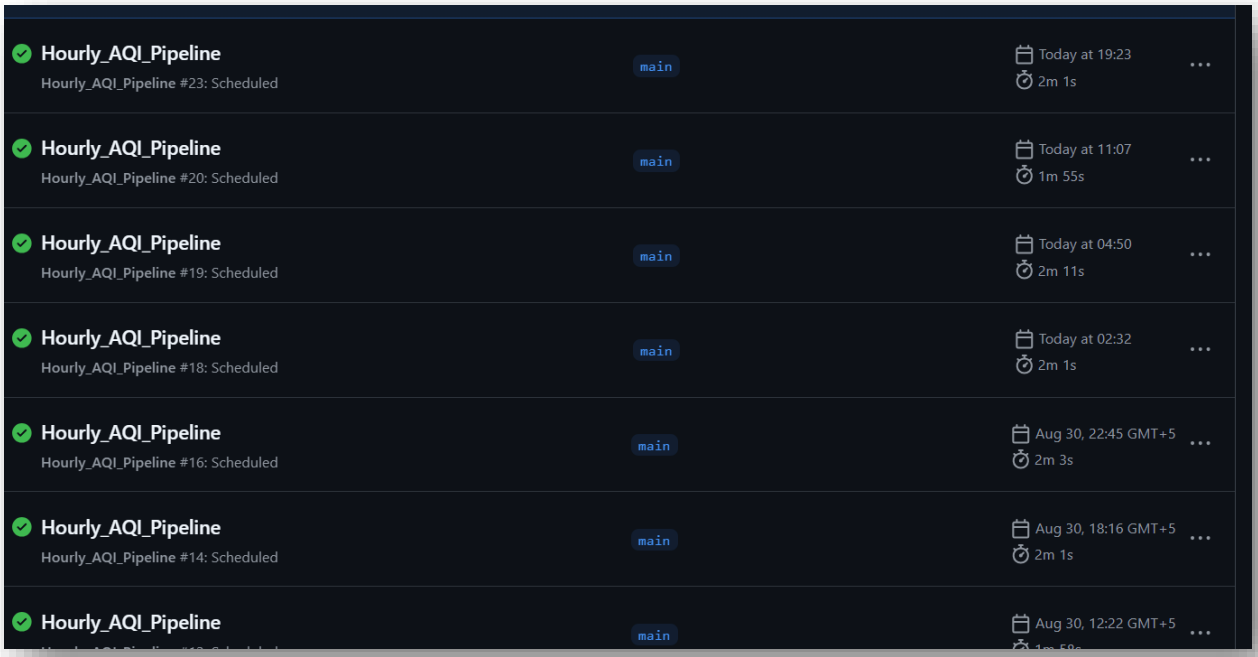
Although the hourly prediction pipeline is configured to run every hour via *GitHub Actions cron* scheduling, *it frequently misses its scheduled executions*.

Reason

This occurs because GitHub Actions does not guarantee precise timing many users run jobs concurrently, causing delays or skipped executions.

Resolution

To mitigate this, the workflow was adjusted to trigger at *the 42nd minute of each hour*, but reliability issues persist.



Hourly Automation Pipeline runs

Potential solution

A potential solution would be to deploy the pipeline on a more robust compute infrastructure (e.g., a dedicated server or cloud VM) to ensure consistent hourly execution, but this was not implemented due to time constraints.

Conclusion

This project successfully developed an end-to-end AQI forecasting system for four Pakistani cities, achieving reliable predictions at 24, 48, and 72-hour horizons. XGBoost, CatBoost, and Random Forest were selected as production models, attaining R^2 values of 0.759, 0.663, and 0.648 respectively. The system integrates automated data collection, feature engineering, cloud storage via Supabase, and model hosting on Hugging Face, with daily monitoring and weekly retraining pipelines ensuring continuous performance. However, GitHub Actions scheduling proved unreliable for hourly execution, and Karachi was excluded due to poor performance. Despite these limitations, the system demonstrates a practical, production-ready solution for multi-horizon AQI forecasting.

References

1. *Open-Meteo. (n.d.). Air Quality API Documentation. Open-Meteo.*

2. *Feast. (n.d.). Feast Feature Store Documentation. Feast.*
3. *Supabase. (n.d.). PostgreSQL Database Documentation. Supabase.*
4. *Hugging Face. (n.d.). Model Hub Documentation. Hugging Face.*
5. *GitHub. (n.d.). GitHub Actions Workflow Documentation. GitHub.*
6. *Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.*
7. *Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. Advances in Neural Information Processing Systems.*
8. *Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.*
9. *Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems.*
10. *Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research.*